# Homework: Data Leakage, Cross-Validation, and Tree-Based Models

GEOL 599 Machine Learning

```
# If needed, install packages:
# install.packages(c("tidyverse", "tidymodels", "ranger", "xgboost"))

library(tidyverse)
library(tidymodels)
library(ranger)
library(xgboost)

tidymodels_prefer()
theme_set(theme_minimal(base_size = 12))

set.seed(7)
```

## Overview

In this homework, you will use the **Forest CoverType** dataset to predict forest cover type from topographic, hydrologic, soil, and wilderness-area variables.

The goals are to:

1. Identify and explain **data leakage**.
2. Compare a leaky model to a properly validated model.
3. Use cross-validation to tune either a **random forest** or an **XGBoost** model.
4. Fit a final model and evaluate it on a held-out test set.
5. Explain the model workflow clearly enough that someone else could trust the results.

The dataset comes from the UCI Machine Learning Repository and contains observations from forested areas in Colorado.

## 1. Load The Data

```
cover_url <- "https://archive.ics.uci.edu/ml/machine-learning-databases/covtype/covtype.data.gz"

cover_names <- c(
  "elevation",
  "aspect",
  "slope",
  "horizontal_distance_to_hydrology",
  "vertical_distance_to_hydrology",
  "horizontal_distance_to_roadways",
  "hillshade_9am",
  "hillshade_noon",
  "hillshade_3pm",
  "horizontal_distance_to_fire_points",
  paste0("wilderness_area_", 1:4),
```

```r
  paste0("soil_type_", 1:40),
  "cover_type"
)

cover_raw <- readr::read_csv(
  cover_url,
  col_names = cover_names,
  show_col_types = FALSE
)

# Use a smaller sample so the assignment runs quickly on laptops.
cover_data <- cover_raw %>%
  mutate(
    cover_type = factor(
      cover_type,
      levels = 1:7,
      labels = c(
        "Spruce/Fir",
        "Lodgepole Pine",
        "Ponderosa Pine",
        "Cottonwood/Willow",
        "Aspen",
        "Douglas-fir",
        "Krummholz"
      )
    )
  ) %>%
  slice_sample(n = 15000)

glimpse(cover_data)
```

```
## Rows: 15,000
## Columns: 55
## $ elevation                          <dbl> 3196, 3088, 2964, 3348, 2542, 3069,~
## $ aspect                             <dbl> 98, 100, 26, 183, 346, 86, 337, 292~
## $ slope                              <dbl> 5, 10, 15, 18, 10, 4, 12, 12, 27, 1~
## $ horizontal_distance_to_hydrology   <dbl> 433, 192, 335, 443, 330, 85, 212, 2~
## $ vertical_distance_to_hydrology     <dbl> 56, 17, 71, 86, 73, 0, 39, 56, 36, ~
## $ horizontal_distance_to_roadways    <dbl> 564, 2652, 182, 4495, 42, 792, 5150~
## $ hillshade_9am                      <dbl> 228, 237, 212, 222, 201, 225, 194, ~
## $ hillshade_noon                     <dbl> 233, 227, 205, 249, 225, 233, 224, ~
## $ hillshade_3pm                      <dbl> 138, 120, 127, 153, 162, 144, 169, ~
## $ horizontal_distance_to_fire_points <dbl> 2524, 2982, 420, 2687, 626, 1448, 4~
## $ wilderness_area_1                  <dbl> 1, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1,~
## $ wilderness_area_2                  <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
## $ wilderness_area_3                  <dbl> 0, 1, 1, 1, 0, 0, 0, 1, 1, 1, 0, 0,~
## $ wilderness_area_4                  <dbl> 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0,~
## $ soil_type_1                        <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
## $ soil_type_2                        <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
## $ soil_type_3                        <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
## $ soil_type_4                        <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0,~
## $ soil_type_5                        <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
## $ soil_type_6                        <dbl> 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,~
## $ soil_type_7                        <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
```

```
## $ soil_type_8                    <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
## $ soil_type_9                    <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
## $ soil_type_10                   <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
## $ soil_type_11                   <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
## $ soil_type_12                   <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
## $ soil_type_13                   <dbl> 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0,~
## $ soil_type_14                   <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
## $ soil_type_15                   <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
## $ soil_type_16                   <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
## $ soil_type_17                   <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
## $ soil_type_18                   <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
## $ soil_type_19                   <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
## $ soil_type_20                   <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
## $ soil_type_21                   <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
## $ soil_type_22                   <dbl> 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1,~
## $ soil_type_23                   <dbl> 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,~
## $ soil_type_24                   <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
## $ soil_type_25                   <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
## $ soil_type_26                   <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
## $ soil_type_27                   <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
## $ soil_type_28                   <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
## $ soil_type_29                   <dbl> 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0,~
## $ soil_type_30                   <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
## $ soil_type_31                   <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
## $ soil_type_32                   <dbl> 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0,~
## $ soil_type_33                   <dbl> 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0,~
## $ soil_type_34                   <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
## $ soil_type_35                   <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
## $ soil_type_36                   <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
## $ soil_type_37                   <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
## $ soil_type_38                   <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
## $ soil_type_39                   <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
## $ soil_type_40                   <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
## $ cover_type                     <fct> Spruce/Fir, Lodgepole Pine, Spruce/~
```

## 2. Explore The Response Variable

```
cover_data %>%
  count(cover_type, sort = TRUE) %>%
  mutate(percent = n / sum(n))
```

```
## # A tibble: 7 x 3
##   cover_type            n percent
##   <fct>             <int>   <dbl>
## 1 Lodgepole Pine     7260 0.484
## 2 Spruce/Fir         5466 0.364
## 3 Ponderosa Pine      892 0.0595
## 4 Krummholz           543 0.0362
## 5 Douglas-fir         491 0.0327
## 6 Aspen               271 0.0181
## 7 Cottonwood/Willow    77 0.00513
```

**Question 1.** Is this a balanced classification problem? Which classes are most common, and which are least common?

Write your answer here:

> It is an imbalanced classification problem. The most common classes are Lodge Pole and Spruce/Fir. The least common is cottonwood/Willow.

## 3. What Is Data Leakage?

Data leakage happens when information reaches the model during training that would **not be available at prediction time**.

Examples include:

- A predictor created directly from the response variable.
- Preprocessing the full dataset before splitting into training and test sets.
- Using future information to predict the past.
- Allowing duplicate or near-duplicate observations to appear in both training and test data.
- Computing group, spatial, or temporal summaries using the full dataset before validation.

In this assignment, we will create an intentionally leaky variable so you can see how leakage can make a model look unrealistically good.

## 4. Create A Leaky Dataset

The variable below is not a real environmental predictor. It is created from the answer we are trying to predict.

```
leaky_data <- cover_data %>%
  mutate(
    leaky_cover_code = as.integer(cover_type)
  )
```

**Question 2.** Why is `leaky_cover_code` an example of data leakage?

Write your answer here:

> It is a created variable, informed by the answer we are predicting. The information in that variable is informed by the environmental variables, so using it would give the model data it shouldn't have access to.

## 5. Split The Leaky Dataset

```
set.seed(7)

leaky_split <- initial_split(
  leaky_data,
  prop = 0.75,
  strata = cover_type
)

leaky_train <- training(leaky_split)
leaky_test <- testing(leaky_split)
```

## 6. Fit A Leaky Model

This model is allowed to use the leaky predictor. Its performance should look suspiciously strong.

```
leaky_recipe <- recipe(cover_type ~ ., data = leaky_train)
```

```r
leaky_model <- rand_forest(
  trees = 200,
  mtry = 8,
  min_n = 5
) %>%
  set_mode("classification") %>%
  set_engine("ranger", importance = "impurity")

leaky_workflow <- workflow() %>%
  add_recipe(leaky_recipe) %>%
  add_model(leaky_model)

leaky_fit <- fit(leaky_workflow, data = leaky_train)

leaky_predictions <- predict(leaky_fit, leaky_test) %>%
  bind_cols(predict(leaky_fit, leaky_test, type = "prob")) %>%
  bind_cols(leaky_test %>% select(cover_type))

leaky_metrics <- metric_set(accuracy, kap, f_meas)

leaky_metrics(
  leaky_predictions,
  truth = cover_type,
  estimate = .pred_class
)
```

```
## # A tibble: 3 x 3
##   .metric  .estimator .estimate
##   <chr>    <chr>          <dbl>
## 1 accuracy multiclass     0.998
## 2 kap      multiclass     0.997
## 3 f_meas   macro          0.978
```

**Question 3.** Why should you distrust the leaky model, even if the accuracy is very high?

Write your answer here:

It's accuracy is almost 1, which is too high to be realistic.

## 7. Build The Proper Dataset

Now remove the leaky variable and restart the modeling process correctly.

```r
model_data <- cover_data %>%
  select(-matches("^leaky_"))

set.seed(7)

cover_split <- initial_split(
  model_data,
  prop = 0.75,
  strata = cover_type
)

cover_train <- training(cover_split)
cover_test <- testing(cover_split)
```

**Question 4.** Why do we split the data before cross-validation, tuning, or final model evaluation?

Write your answer here:

To prevent data leakage.

## 8. Cross-Validation Folds

Cross-validation should be created from the **training data only**.

```
set.seed(7)

cover_folds <- vfold_cv(
  cover_train,
  v = 5,
  strata = cover_type
)

cover_folds
```

```
## #  5-fold cross-validation using stratification
## # A tibble: 5 x 2
##   splits              id
##   <list>              <chr>
## 1 <split [8998/2251]> Fold1
## 2 <split [8999/2250]> Fold2
## 3 <split [8999/2250]> Fold3
## 4 <split [9000/2249]> Fold4
## 5 <split [9000/2249]> Fold5
```

## 9. Choose Your Model

Choose **one** model:

- `"random_forest"`
- `"xgboost"`

**Question 5.** Which model did you choose, and why? Your reason can be based on interpretability, speed, expected performance, or what you want to practice.

Write your answer here:

I chose random forest in hopes it runs faster than xgboost.

## 10. Create The Recipe

The predictors are already numeric, including the wilderness-area and soil-type indicator variables. We will still include a median imputation step as a best practice.

```
cover_recipe <- recipe(cover_type ~ ., data = cover_train) %>%
  step_impute_median(all_numeric_predictors())

cover_recipe
```

```
##
## -- Recipe ----------------------------------------------------------------------
##
```

```
## -- Inputs

## Number of variables by role

## outcome:    1
## predictor: 54

##

## -- Operations

## * Median imputation for: all_numeric_predictors()
```

## 11. Define Candidate Models

```
rf_spec <- rand_forest(
  trees = tune(),
  mtry = tune(),
  min_n = tune()
) %>%
  set_mode("classification") %>%
  set_engine("ranger", importance = "impurity")

xgb_spec <- boost_tree(
  trees = tune(),
  tree_depth = tune(),
  learn_rate = tune(),
  loss_reduction = tune(),
  sample_size = tune(),
  mtry = tune(),
  min_n = tune()
) %>%
  set_mode("classification") %>%
  set_engine("xgboost", eval_metric = "mlogloss")
```

## 12. Create The Workflow

```
cover_workflow <- workflow() %>%
  add_recipe(cover_recipe) %>%
  add_model(rf_spec)

cover_workflow
```

```
## == Workflow ====================================================================
## Preprocessor: Recipe
## Model: rand_forest()
##
## -- Preprocessor ----------------------------------------------------------------
## 1 Recipe Step
##
## * step_impute_median()
##
## -- Model -----------------------------------------------------------------------
## Random Forest Model Specification (classification)
##
## Main Arguments:
```

```
##   mtry = tune()
##   trees = tune()
##   min_n = tune()
##
## Engine-Specific Arguments:
##   importance = impurity
##
## Computational engine: ranger
```

## 13. Create A Small Tuning Grid

For homework, use a modest grid so the code finishes in a reasonable amount of time. You may expand the grid if your computer can handle it.

```
tune_grid_values <- grid_regular(  trees(range = c(300, 500)),
  mtry(range = c(2, 6)),
  min_n(range = c(4, 8)),
  levels = c(trees = 2, mtry = 3, min_n = 3))

tune_grid_values
```

```
## # A tibble: 18 x 3
##    trees  mtry min_n
##    <int> <int> <int>
##  1   300     2     4
##  2   500     2     4
##  3   300     4     4
##  4   500     4     4
##  5   300     6     4
##  6   500     6     4
##  7   300     2     6
##  8   500     2     6
##  9   300     4     6
## 10   500     4     6
## 11   300     6     6
## 12   500     6     6
## 13   300     2     8
## 14   500     2     8
## 15   300     4     8
## 16   500     4     8
## 17   300     6     8
## 18   500     6     8
```

## 14. Tune With Cross-Validation

```
classification_metrics <- metric_set(accuracy, kap, f_meas)

set.seed(7)

cv_results <- tune_grid(
  cover_workflow,
  resamples = cover_folds,
  grid = tune_grid_values,
  metrics = classification_metrics,
```

```
  control = control_grid(save_pred = TRUE)
)

collect_metrics(cv_results)
```

```
## # A tibble: 54 x 9
##     mtry trees min_n .metric  .estimator  mean     n std_err .config
##    <int> <int> <int> <chr>    <chr>      <dbl> <int>   <dbl> <chr>
##  1     2   300     4 accuracy multiclass 0.663     5 0.00419 pre0_mod01_post0
##  2     2   300     4 f_meas   macro      0.615     5 0.0304  pre0_mod01_post0
##  3     2   300     4 kap      multiclass 0.418     5 0.00801 pre0_mod01_post0
##  4     2   300     6 accuracy multiclass 0.666     5 0.00535 pre0_mod02_post0
##  5     2   300     6 f_meas   macro      0.652     5 0.00699 pre0_mod02_post0
##  6     2   300     6 kap      multiclass 0.422     5 0.00945 pre0_mod02_post0
##  7     2   300     8 accuracy multiclass 0.664     5 0.00162 pre0_mod03_post0
##  8     2   300     8 f_meas   macro      0.651     5 0.00477 pre0_mod03_post0
##  9     2   300     8 kap      multiclass 0.420     5 0.00289 pre0_mod03_post0
## 10     2   500     4 accuracy multiclass 0.661     5 0.00415 pre0_mod04_post0
## # i 44 more rows
```

**Question 6.** Which metric do you think is most useful for this dataset: accuracy, kappa, or F-measure? Explain your choice.

Write your answer here:

> Kappa is the most informative because it adjusts the accuracy for what could happen by random chance.

## 15. Select The Best Model

```
best_params <- select_best(cv_results, metric = "f_meas")

best_params
```

```
## # A tibble: 1 x 4
##    mtry trees min_n .config
##   <int> <int> <int> <chr>
## 1     2   300     6 pre0_mod02_post0
```

```
final_workflow <- finalize_workflow(
  cover_workflow,
  best_params
)

final_workflow
```

```
## == Workflow ====================================================================
## Preprocessor: Recipe
## Model: rand_forest()
##
## -- Preprocessor ----------------------------------------------------------------
## 1 Recipe Step
##
## * step_impute_median()
##
## -- Model -----------------------------------------------------------------------
```

```
## Random Forest Model Specification (classification)
##
## Main Arguments:
##   mtry = 2
##   trees = 300
##   min_n = 6
##
## Engine-Specific Arguments:
##   importance = impurity
##
## Computational engine: ranger
```

**Question 7.** What hyperparameters were selected for your model? Do the selected values suggest a simple model or a more flexible model?

Write your answer here:

> 300 trees were selected with 6 iterations.

## 16. Final Test Evaluation

Only use the test set once, after model selection is complete.

```
final_fit <- last_fit(
  final_workflow,
  split = cover_split,
  metrics = classification_metrics
)

collect_metrics(final_fit)
```

```
## # A tibble: 3 x 4
##   .metric  .estimator .estimate .config
##   <chr>    <chr>          <dbl> <chr>
## 1 accuracy multiclass     0.675 pre0_mod0_post0
## 2 kap      multiclass     0.439 pre0_mod0_post0
## 3 f_meas   macro          0.666 pre0_mod0_post0
```

```
final_predictions <- collect_predictions(final_fit)

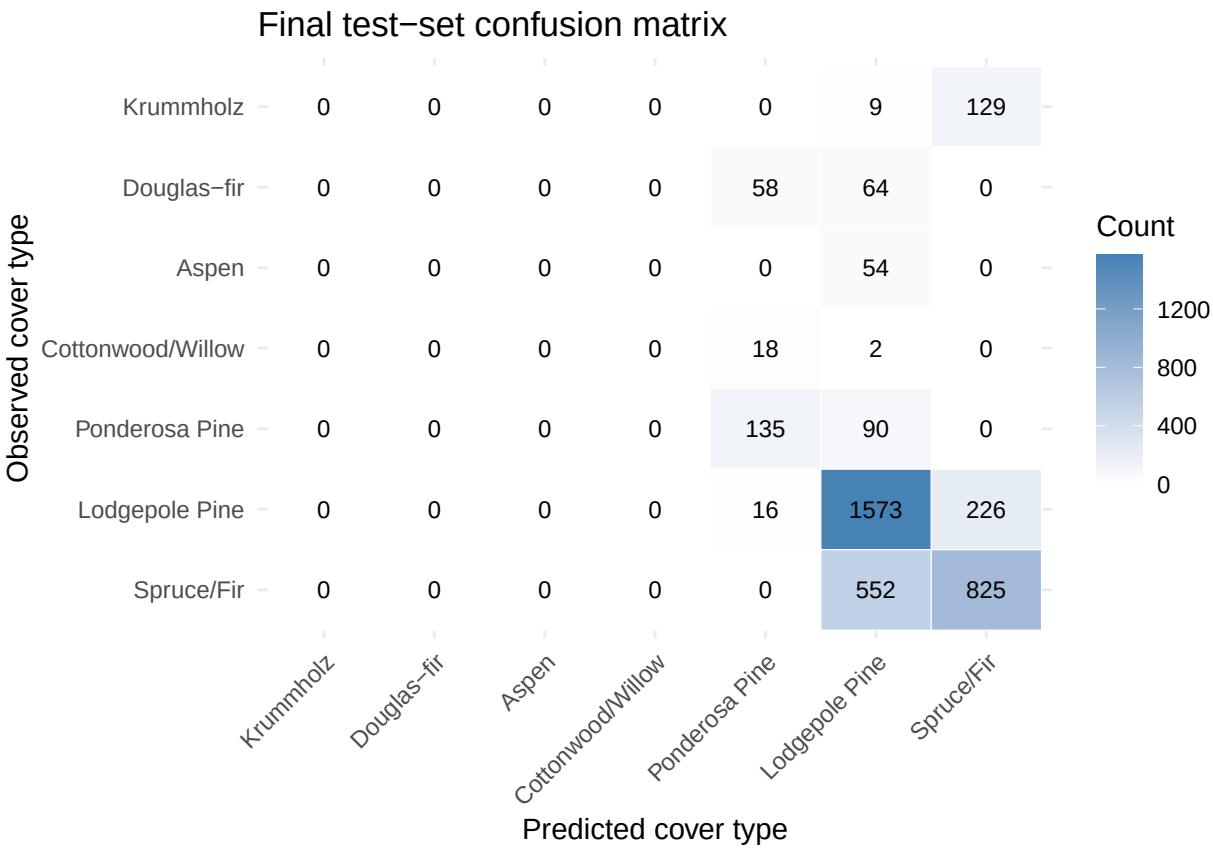
final_predictions %>%
  conf_mat(truth = cover_type, estimate = .pred_class)
```

```
##                   Truth
## Prediction          Spruce/Fir Lodgepole Pine Ponderosa Pine Cottonwood/Willow
##   Spruce/Fir               825           226              0                 0
##   Lodgepole Pine           552          1573             90                 2
##   Ponderosa Pine             0            16            135                18
##   Cottonwood/Willow          0             0              0                 0
##   Aspen                      0             0              0                 0
##   Douglas-fir                0             0              0                 0
##   Krummholz                  0             0              0                 0
##                   Truth
## Prediction          Aspen Douglas-fir Krummholz
##   Spruce/Fir            0           0       129
##   Lodgepole Pine       54          64         9
##   Ponderosa Pine        0          58         0
```

```
##   Cottonwood/Willow     0           0         0
##   Aspen                 0           0         0
##   Douglas-fir           0           0         0
##   Krummholz             0           0         0
```

```r
confusion_plot_data <- final_predictions %>%
  count(cover_type, .pred_class, name = "n") %>%
  complete(cover_type, .pred_class, fill = list(n = 0))

ggplot(confusion_plot_data, aes(x = .pred_class, y = cover_type, fill = n)) +
  geom_tile(color = "white") +
  geom_text(aes(label = n), size = 3) +
  scale_fill_gradient(low = "white", high = "steelblue") +
  scale_x_discrete(limits = rev) +
  labs(
    title = "Final test-set confusion matrix",
    x = "Predicted cover type",
    y = "Observed cover type",
    fill = "Count"
  ) +
  theme_minimal(base_size = 11) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```




**Question 8.** Which classes does the model predict well? Which classes are most often confused?

Write your answer here:

> Krummholz is predicted wrong the most, and confused with Spruce/Fir. Lodgepole and Spruce/Fir also get confused with each other. Lodgepole pine was predicted the most correctly overall.

## 17. Compare Leaky And Proper Results

```
proper_test_metrics <- collect_metrics(final_fit) %>%
  mutate(workflow = "Proper workflow")

leaky_test_metrics <- leaky_metrics(
  leaky_predictions,
  truth = cover_type,
  estimate = .pred_class
) %>%
  mutate(workflow = "Leaky workflow")

bind_rows(leaky_test_metrics, proper_test_metrics) %>%
  select(workflow, .metric, .estimate)
```

```
## # A tibble: 6 x 3
##   workflow        .metric  .estimate
##   <chr>           <chr>        <dbl>
## 1 Leaky workflow  accuracy     0.998
## 2 Leaky workflow  kap          0.997
## 3 Leaky workflow  f_meas       0.978
## 4 Proper workflow accuracy     0.675
## 5 Proper workflow kap          0.439
## 6 Proper workflow f_meas       0.666
```

**Question 9.** Compare the leaky model results to the proper final model results. What does this comparison teach you about data leakage?

Write your answer here:

> Data leakage will fill you with optimisim that you have a good model, but then will let you down because it is all a lie. A proper model that the data is split properly will have much lower performance metric.

## 18. Simple Permutation Importance

This section gives a model-agnostic way to ask: how much worse does the model get if one predictor is randomly shuffled?

```
final_trained_workflow <- fit(final_workflow, data = cover_train)

baseline_predictions <- predict(final_trained_workflow, cover_test) %>%
  bind_cols(cover_test %>% select(cover_type))

baseline_f <- f_meas(
  baseline_predictions,
  truth = cover_type,
  estimate = .pred_class
) %>%
  pull(.estimate)

set.seed(7)

importance_variables <- c(
  "elevation",
  "slope",
```

```
  "horizontal_distance_to_hydrology",
  "horizontal_distance_to_roadways",
  "horizontal_distance_to_fire_points"
)

permutation_importance <- map_dfr(importance_variables, function(var_name) {
  permuted_test <- cover_test
  permuted_test[[var_name]] <- sample(permuted_test[[var_name]])

  permuted_predictions <- predict(final_trained_workflow, permuted_test) %>%
    bind_cols(cover_test %>% select(cover_type))

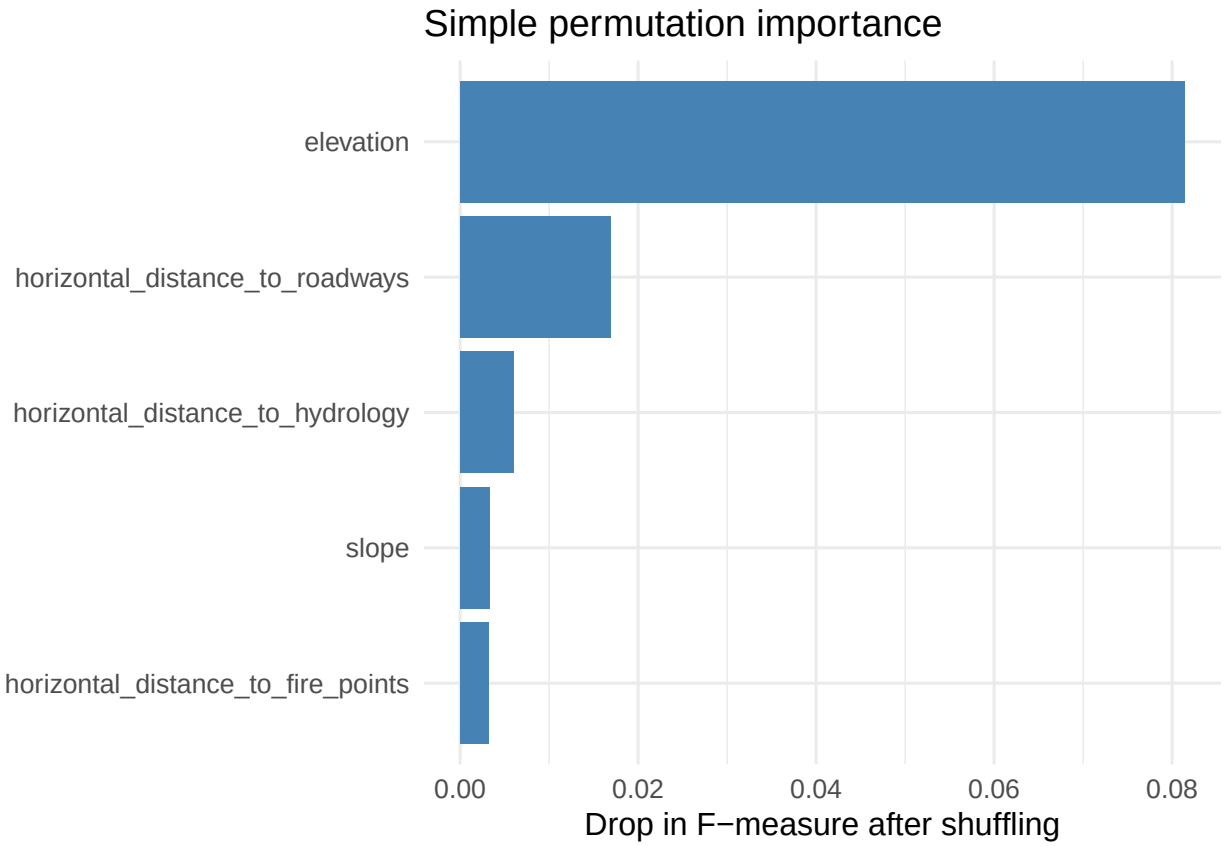
  permuted_f <- f_meas(
    permuted_predictions,
    truth = cover_type,
    estimate = .pred_class
  ) %>%
    pull(.estimate)

  tibble(
    variable = var_name,
    baseline_f_meas = baseline_f,
    permuted_f_meas = permuted_f,
    importance = baseline_f - permuted_f
  )
})

permutation_importance %>%
  arrange(desc(importance))
```

```
## # A tibble: 5 x 4
##   variable                           baseline_f_meas permuted_f_meas importance
##   <chr>                                        <dbl>           <dbl>      <dbl>
## 1 elevation                                    0.667           0.586    0.0814
## 2 horizontal_distance_to_roadways              0.667           0.650    0.0169
## 3 horizontal_distance_to_hydrology             0.667           0.661    0.00606
## 4 slope                                        0.667           0.664    0.00327
## 5 horizontal_distance_to_fire_points           0.667           0.664    0.00319
```

```
permutation_importance %>%
  mutate(variable = fct_reorder(variable, importance)) %>%
  ggplot(aes(x = importance, y = variable)) +
  geom_col(fill = "steelblue") +
  labs(
    title = "Simple permutation importance",
    x = "Drop in F-measure after shuffling",
    y = NULL
  )
```




**Question 10.** Which variables appear most important in your model? Does that make environmental sense?

Write your answer here:

> Elevation is the most important in the model, and then hortizontal distance to roadways.

## 19. Final Reflection

Answer the following in a short paragraph.

**Question 11.** Describe your complete modeling workflow. Your answer should mention:

- How you avoided data leakage.
- Why you used cross-validation.
- Which model you chose.
- Which metric you used to select the model.
- How the final test performance compared to the leaky model.
- One limitation of this modeling approach.

Write your answer here:

> Data leakage was prevented by splitting the data before training the model. Cross validation occured with 5 folds stratified by cover type. I chose a random forest model because I wanted to try the most parsimonious option first. The model was selected usign the f-measure. the final test performance seemed dismal compared to the leaky model. I do not think I chose the proper number of trees in the grid, so maybe I needed to try the xgboost method. Right now, the model I built I would not say is particularly useful.